

CS2030S Midterms Cheatsheet

by @jmsandiegoo

2. Variable Type

Data Abstraction: Variables

A variable is an abstraction that allows us to give a user-friendly name to a piece of data in memory

Static Type

Variables type are declared during compile time. A variable can only hold values of same type and cannot be changed.

Primitive Types & Subtyping

```
byte (0) <: short (0) <: int (0) <: long (0L)
<: float (0.0f) <: double (0.0d)
```

```
char ('\u0000') <: int
```

```
boolean (false)
```

** Primitive type never share their value with each other*

Reference Types

```
classes (null)
```

** Reference type share their value with each other*

3. Functions

Abstraction (Computation) Function

Function allows:

- Compartmentalize computation
 - complexity and variables only within the function body and doesn't affect outside code
- Hide implementation
 - caller only need to worry what the function does (parameters and return)
 - does not affect caller when code evolves and changes overtime
- Reduce repetition
 - can just call the function with varying parameters to conduct same computation for diff values
 - also any changes can be done in one place which is the function itself

Abstraction Barrier

A barrier between the code that calls (**by: client**) the function and the code that implements/define the function (**by: implementer**).

4. Encapsulation

Abstraction: Composite Data Types

Group primitive types together and abstracts it away allowing us to think at higher level of conceptualization

```
typedef struct {
    double x, y; // coordinate of center.
    double r; // radius
} circle;
```

** Usually has corresponding functions that uses the struct to completely be shielded from the representation*

Abstraction: Class a.k.a Encapsulation

Abstracts composite type and associated functions on the same side of abstraction barrier
Consists of data fields/members/states/attributes & methods

```
class Circle {
    double x;
    double y;
    double r;

    double getArea() {
        return 3.141592653589793 * r * r;
    }
}
```

5. Information Hiding

Use of access modifiers to prevent field or method access from outside class / abstraction barrier

Constructors

```
// inside Circle class
public Circle(double x, double y, double r) {
    this.x = x;
    this.y = y;
    this.r = r;
}
// outside
Circle c = new Circle(0.0, 0.5, 10.0);
```

6. Tell Don't Ask

Accessor / Getter & Mutator / Setter

A public method in class that retrieves and updates the value of the field.

** Harmful, exposes internal representation. Still can be used in "no choice" scenarios*

Tell Don't Ask Principle

Client tells the circle what to do, instead of asking for its fields

```
// wrong
double cX = c.getX();
double cY = c.getY();
double r = c.getR();
boolean isInCircle =
    ((x - cX) * (x - cX) +
     (y - cY) * (y - cY)) <= r * r;
// correct
boolean isInCircle = c.contains(x, y);
```

7-8. Class Fields & Methods

Class Fields

Useful for sharing shared values across class instances

```
class Math {
    :
    public static final double PI = 3.141...;
    :
}
// access
Math.PI;
```

** final means cannot be changed / constant*

Class Methods

A method that is independent of an instance. Has no **this** reference

```
class Circle {
    :
    // can only access static fields
    public static int getNumOfCircles() {
        return Circle.lastId;
    }
    :
}

// main program entry point
public final static void main(String[] args) {
    :
}
```

9. Composition

Models a HAS-A relationship, enables further layer of abstraction and maintainability.

```
class Circle {
    private Point c; // HAS-A Point (center)
    private double r; // the length of radius
    :
}
```

HAS-A can have shared

10. Heap & Stack

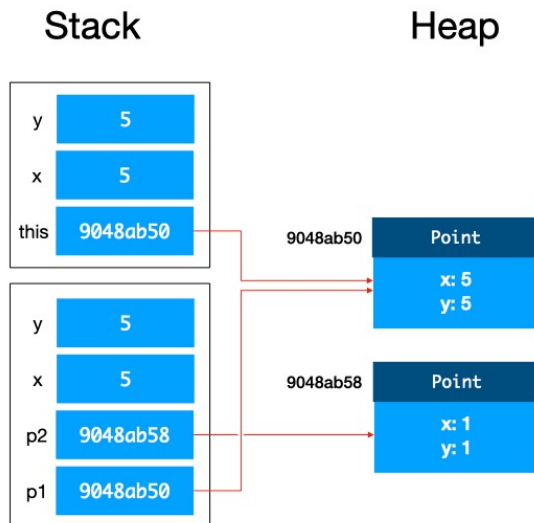
Type of Calls

Primitive Types: call by value

Reference Types: call by reference

JVM Memory partitions:

- **method area:** stores method codes
- **metaspace:** stores meta information of class
- **heap:** stores dynamically allocated objects
- **stack:** stores local variables



* Don't forget label the stack frames & the main method args variable and array

11 - 13. Inheritance, Overriding & Overloading

Inheritance

Models a IS-A relationship, enables classes to have a parent-child relationship.

- **Child (subclass):** has all the parent's and its own unique characteristics (extends)
- **Parent (parent/super class):** the more general object where the child is basing from

```
// version 0.3 (using inheritance)
class ColoredCircle extends Circle {
    private Color color;

    public ColoredCircle(Point center,
        double radius,
        Color color) {
        // call the parent's constructor
        super(center, radius);
        this.color = color;
    }
}
```

Run-Time vs Compile-Time Type

```
Circle c = new ColoredCircle(p, 0, blue);
```

Compile-time Type is the type of the var c during compilation which is Circle

Run-Time Type is the type of the var c during runtime which is ColoredCircle

* All classes inherits from Object

Overriding

Overriding allows child class to change the inherited method behaviour.

Method Signature: method name and the number, type, and order of its parameters

Method Descriptor: method signature + return type

```
class Circle {
    :
    // hints compiler overriding occurs
    // so compiler will help us check
    // also return type can subtype
    @Override
    public String toString() {
        return "center: " +
            this.c + ", radius: " + this.r + " ";
    }
}
```

* super can be used to call overridden methods

Overloading

Allows to define multiple methods of same name but different method signature. Also

* Cannot overload with different return type

14. Polymorphism

Method overriding and dynamic typing allows polymorphism to occur. An obj can have many forms (takes form of its many subtypes) in which its method is invoked depending run-time type.

```
// version 0.1 (with polymorphism)
boolean contains(Object[] array, Object obj) {
    for (Object curr : array) {
        // Polymorphism
        if (curr.equals(obj)) {
            return true;
        }
    }
    return false;
}
```

15 - 16. Method Invocation & LSP

Method Invocation

Identifies which method implementation to be executed. Two step process:

1. **Compile Time:** Identify the target's compile-time type, find and store closest method descriptor for the method invoked.
* choose the most specific method that accepts $N \leq M$
2. **Run Time:** Identify target's run-time type and search from run-time type up the hierarchy. Execute the first matched method.

* class methods don't apply here

Liskov Substitution Principle

$S \leq T$, states that code using type T if substituted with S should still work and pass the test cases. S should not change the overall behaviour of T

* use final for class and method to prevent inheritance and overriding

17 - 18. Abstract Class & Interface

Abstract class

A class that is too general to have all method defined
It cannot be instantiated
Not all methods have to be abstract

```
abstract class Shape {
    :
    // subclasses must implement this
    abstract public double getArea();
}
```

Interface

Models "can do" behaviour. To group classes not necessarily "IS-A" but with same "do" behaviour.

- Class can only extend from one class but can implement multiple interfaces
- Interface can extend from multiple interfaces but not from class

```
interface GetAreable {
    public abstract double getArea();
}
```

```
abstract class Shape implements GetAreable {
    :
```

```
class Flat extends RealEstate
    implements GetAreable {
    :
    @Override
    public double getArea() {
        :
    }
}
```

Interface Casting

```
interface I {
    :
}
class A {}
class B implements I {}
// Compiles, widening type conversion
I i1 = new B();
// Also compiles, as subtype could implement
// interface
I i2 = (I) new A();
```

19. Wrapper Class

Wraps primitive types to its corresponding class.

```
Integer i = new Integer(4)
int j = i.intValue();
// with auto-boxing/unboxing
Integer i = 4;
int j = i;
```

** less performant terms of heap space and immutability as a result of auto-box*

20 - 21. Casting & Variance

Casting

- **Widening type conversion:** done implicitly
- **Narrowing type conversion:** explicit and must be careful of as it could cause run-time errors

```
// Circle <: GetAreable
// ok
GetAreable ga = findLargest(circles);
// ok
Circle c2 = (Circle) findLargest(circles);
```

Variance

Refers to the subtyping relationship between complex types like arrays

- **covariant:** $S <: T$ implies $C(S) <: C(T)$
- **contravariant:** $S <: T$ implies $C(T) <: C(S)$
- **invariant:** neither of the two

** Java array is covariant*

22. Exception

Catching exception

```
try {
    // do something
    reader = new FileReader(filename);
    scanner = new Scanner(reader);
    value = scanner.nextInt();
}
// (Exception1 | Exception2 e)
catch (FileNotFoundException e) {
    // handle exception
    System.err.println("Unable to open");
}
finally {
    // clean up code
    // would still run even after
    // return or throw
    if (scanner != null)
        scanner.close();
}
```

Throwing / Passing Exception

```
// throws after params
public Circle(Point c, double r)
    throws IllegalArgumentException {
    if (r < 0) {
        throw new IllegalArgumentException(
            "radius cannot be negative.");
    }
    this.c = c;
    this.r = r;
}
// can also catch it inside function
// if just pass the buck no throw would occur
// just throws keyword after function params
```

Checked vs Unchecked

Checked: an exception that a programmer has no control over with despite no error in code. MUST BE CHECKED AND HANDLED if not won't compile.

Unchecked <: RuntimeException: exception caused my programmer's mistake in the code. NOT EXPLICITLY CAUGHT / THROWN. Leads to runtime errors

Control Flow

When an exception is thrown instruction after it will not be executed. Instead the caller that catch and handles the error will be passed the control flow of the program.

Creating Own Exception

Own excpetion can be created by extending the existing Java Exception classes.

Override Method Throwing Exception

Can be overridden but make sure the method throws the same or a subclass of the exception thrown

Best Practices

- Catch Exceptions to Clean Up
- Do not catch-them-all!
- Overreacting: don't terminate when exception is caught
- Do Not Break Abstraction Barrier
- Do NOT Use Exception As a Control Flow Mechanism

23 - 24. Generics & Type Erasure

Generic Class

```
// <> type parameter
// compiler can also type check
class Pair<S,T> {
    private S first;
    private T second;

    public Pair(S first , T second) {
        this.first = first;
        this.second = second;
    }

    S getFirst() {
        return this.first;
    }
    T getSecond() {
        return this.second;
    }
}
// passing T parameter to another generic
class DictEntry<T> extends
    Pair<String,T> {
    :
}
// once type args passed it is now
// parameterized type
Pair<String,Integer> p =
    new Pair<String,Integer>("hello", 4);
```

Bounded Parameters & Generic Methods

```
class A {
    // version 0.5
    // extends even supertype is interface
    // extends also works if T is GetAreable
    public static <T extends GetAreable>
        T findLargest(T[] array) {
        double maxArea = 0;
        T maxObj = null;
        for (T curr : array) {
            double area = curr.getArea();
            if (area > maxArea) {
                maxArea = area;
                maxObj = curr;
            }
        }
        return maxObj;
    }
}
```

* bounded params can be used for declaring generic classes as well

Type Erasures

Java takes a code sharing approach, it erase the type parameters and type arguments during compilation (after type checking).

```
class Pair {
    private Object first;
    private Object second;

    public Pair(Object first , Object second) {
        this.first = first;
        this.second = second;
    }

    Object getFirst() {
        return this.first;
    }

    Object getSecond() {
        return this.second;
    }
}
```

Arrays and Generics don't mix

Leads to heap pollution as generics is not reliable

```
// create a new array of pairs
Pair<String,Integer>[] pairArray =
    new Pair<String,Integer>[2];

// pass around the array of pairs
// as an array of object
Object[] objArray = pairArray;

// put a pair into the array
// — no ArrayStoreException!
objArray[0] =
    new Pair<Double,Boolean>(3.14, true);

// Realistically these are illegal
new Pair<S,T>[2];
new T[2];
```

25. Unchecked Warnings

Unchecked warning means that compiler could not help us detect possible run-time exceptions (ClassCastException) due to type erasures.

```
class Array<T> {
    private T[] array;

    Array(int size) {
        // leave a reason note
        // min scope and only declarations
        // ensure no error would occur
    }
}
```

```
@SuppressWarnings("unchecked")
// throws unchecked as it may have
// items that are not type T
T[] a = (T[]) new Object[size];
this.array = a;
}

public void set(int index, T item) {
    this.array[index] = item;
}

public T get(int index) {
    return this.array[index];
}
}
```

Raw Types

Using generic types without type arguments will also lead to run-time errors.

26. Wildcards

Upper-Bounded Wildcards

```
// ? can be substituted with S
// or subtype of it
```

```
Array<? extends S>
```

```
// upper-bound is covariant
Array<C> <: Array<? extends C>
```

```
C <: S, Array<? extends C> <: Array<? extends S>
```

```
Array<C> <: Array<? extends S> (transitive)
```

Lower-Bounded Wildcards

```
// ? can be substituted with S
// or supertype of it
```

```
Array<? super S>
```

```
// lower-bound contravariant
Array<S> <: Array<? super S>
C <: S, Array<? super S> <: Array<? super C>
Array<S> <: Array<? super C> (transitive)
```

Producer Extends Consumer Super (PECS)

Producer: returns variable of type T <? extends T>
Consumer: takes in a variable of type T <? super T>

Unbounded Wildcards

Supertype of all parameterized type

```
// ? can be substituted with any type
// array of objects of some specific ,
// but unknown type
Array<?>
```

```
// array of object , with
// typechecking by the compiler
Array<Object>
```

```
// array of Object instances ,
// without type checking.
Array
```

```
// Can be used with instanceof
a instanceof A<?>
// Can be used with array
T[] a = (T[]) new Comparable<?>[size]
```

27. Type Inference

Diamond Operator

Java helps infer the parameter types of the generic when instantiating a new instance.

```
// inferred the new instance as
// type Pair<String, Integer>
Pair<String, Integer> p = new Pair<>();
```

Target Typing

1. Identify Target Typing
2. Identify Argument Typing
3. Bounds on Typing

** Lowest Bound is taken*